Caminos mínimos: Dijkstra

El primer algoritmo que vamos a ver es el problema de caminos mínimos que dice: dado un grafo y un nodo particular del mismo, buscar el costo del camino más corto entre ese nodo y todos los demás, siempre que exista un camino. El resultado es un grafo con los mismos nodos que el original pero sólo hay aristas entre el nodo buscado y todos los demás y el peso de la arista es el costo del camino mínimo. No devuelve el camino sino el costo. Si quisiéramos además recuperar el camino, se debería modificar el algoritmo para que lo vaya almacenando. Por este mismo motivo, si hubiera más de un camino con igual costo, no afecta al algoritmo.

Para poder ejecutar el algoritmo es necesario contar con un grafo dirigido, con todas sus aristas con peso positivo. Esto no significa que el grafo deba ser conexo, es decir, puede haber pares de nodos que no estén unidos por una arista, pero si ésta existe, debe tener un peso positivo.

El conjunto de candidatos es el conjunto de vértices del grafo, sacando el nodo de origen. En cada iteración, el método de selección será el encargado de elegir de todos los nodos pendientes de visitar, aquel que tenga el camino de menor costo desde el origen y lo marca como visitado, además de confirmar como definitivo el costo del camino mínimo desde el origen hasta dicho nodo.

Para lograr esto, el algoritmo tendrá actualizados los costos del camino mínimo encontrado hasta el momento entre el origen y todos los nodos pendientes. Para comenzar, sólo tendrá como información que el costo del camino mínimo entre el nodo origen y todos los demás es el peso de la arista directa que hay en el grafo de origen desde el origen a los demás nodos. Luego, cada vez que se seleccione un nodo, se analiza si utilizando como puente dicho nodo, se puede mejorar el camino entre el origen y el resto de los pendientes. Cuando se hayan visitado todos los nodos, el algoritmo habrá finalizado.

Seudocodigo

Algoritmo Dijkstra Entrada: G: Grafo<entero>, v: entero Salida: A Grafo<entero>

```
//Paso 1
//Conjunto de Vértices ya visitados
Conjunto<entero> Visitados ← {v}

//Paso 2
//Grafo auxiliar
Grafo<entero> A ← inicializarGrafo();
para cada w ∈ Vertices(G)
   agregarVertice(A, w)
fin para
para cada v' ∈ Adyacentes(G, v)
   agregarArista(A, v, v', Peso(G, v, v'))
```

```
fin para

//Conjunto de Vértices pendientes de calculo
Pendientes ← Vertices(G) \ Visitados

//Paso 3
mientras Pendientes ≠ ∅
   w ← n : PesoArista(A, v, n) = mín{PesoArista(A, v, p)} ∀ n, p ∈ Pendientes
   Visitados ← Visitados ∪ {w}
   Pendientes ← Pendientes \ {w}
   auxPendientes ← Pendientes

   mientras auxPendientes ≠ ∅
      p ← elegir(auxPendientes)
      auxPendientes ← auxPendientes \ {p}

      si existeArista(A, v, w) Y existeArista(G, w, p)
         si existeArista(A, v, p)
            si pesoArista(A, v, w) + pesoArista(G, w, p) < pesoArista(A, v, p)
               agregarArista(A, v, p, pesoArista(A, v, w) + pesoArista(G, w, p))
            fin si
         sino
            agregarArista(A, v, p, pesoArista(A, v, w) + pesoArista(G, w, p))
         fin si
      fin si
   fin mientras
fin mientras

devolver A
```

Codigo

```
1. class Nodo {
2.       boolean selec
3.       int costo
4.       int selec
5. }
6. algoritmo Dijkstra (G : GrafoTDA) { // El nodo base es el 0
7.        Nodo[ ] U = crearArreglo(G) // El arreglo inicial
```

```
8.      while (!completo(U)) { // El ciclo externo
9.              int i = buscarMinimo(U) // Mínima distancia
10.             U[i] = true // El nodo está visitado
11.             actualizar(G, U, i) // El ciclo interno
12.     }
13.     return U
14. }

1.Nodo[ ] crearArreglo (Nodo[ ] A) { // Crea el arreglo inicial
2.      int cant = A.vertices()
3.      Nodo[ ] Arreglo = new Nodo[cant]
4.      for (int i = 0; i < cant; i++) {
5.              Nodo O = new Nodo()
6.              O.prev = 0 // Al inicio, todos apuntan a 0
7.              if (i == 0)
8.                      O.selec = true // Se selecciona
9.                      O.costo = 0 // El costo de 0 es 0
10.             } else {
11.                     O.selec = false // No se selecciona
                        if (existeArista (0, i))
13.                             O.costo = 0 // El costo desde 0
14.                     else
15.                             O.costo = ∞ // Costo infinito
16.                     }
17.             Arreglo[i] = O
18.     }
19.     return Arreglo
20. }

1. boolean completo (Nodo[ ] X) { // ¿Todos visitados?
2.      boolean listo = true
3.      for (int i = 0; i < A.length; i++)
4.              listo = listo && X.selec
5.      return listo
6. }




7. int buscarMinimo (Nodo[ ] E) { // Distancia mínima
8.      int min = ∞
9.      int inx =0
10.     for (int i = 0; i < A.length; i++) {
11.             if (E[i].costo < min && !A.selec)
12.                     min = E[i].costo // Nuevo mínimo
13.                     inx = i
14.              }
15.      }
```

```
16.       return inx
17. }

// Aquí se realiza el ciclo interno
1. void actualizar (GrafoTDA[ ] G, Nodo[ ] A, int i) {
2.        for (int j = 0; j < A.length; j++) {
3.                  if (! A[j].selec && G.existeArista (i, j))
// Aquí se determina si se actualizan o no los valores del nodo
4.                            if (A[i].costo + G.pesoArista (i, j) < A[j].costo)
5.                                      A[j].costo = A[i].costo + G.pesoArista (i, j)
6.                                      A[j].prev = i
7.                             }
8.                  }
9.        }
10. }
```